

# Variables y Tipos de datos JavaScript Guia

## Variables y Tipos de Datos en JavaScript

**Nota:** Este documento cubre los conceptos fundamentales de variables, tipos de datos primitivos y estructuras de datos en JavaScript.

### 1. Variables

Una **variable** es una ubicación de almacenamiento con nombre para datos.

#### Declaración de Variables

En JavaScript, las variables se pueden declarar usando `let` , `const` o `var` .

```
let nombre = "Pepe";
console.log(nombre);

let edad = 20;
console.log(edad);
```

| Palabra clave | ¿Se puede reasignar? | ¿Cuándo usarla?                          |
|---------------|----------------------|------------------------------------------|
| let           | Sí                   | Cuando el valor va a cambiar             |
| const         | No                   | Cuando el valor NO va a cambiar          |
| var           | Sí                   | Forma antigua. Evítala en código moderno |

### 2. Operadores Aritméticos

```
let suma = 10 + 5;           // 15
let resta = 10 - 5;          // 5
```

```
let multiplicacion = 10 * 5; // 50
let division = 10 / 5;      // 2
let residuo = 10 % 3;       // 1 - Módulo (residuo de la división)
let potencia = 2 ** 3;      // 8 - Potencia (2 elevado a 3)

console.log("El resultado de la división es: " + division);
```

## Ejemplo 1: Total de Compra

```
let precioPan = 500;
let precioLeche = 4000;

let totalCompra = precioPan + precioLeche;

console.log("El total de la compra es: ", totalCompra);
```

## Ejemplo 2: Cuenta del Restaurante con Propina

```
let costoComida = 45000;
let porcentajePropina = 10;

let valorPropina = (costoComida * porcentajePropina) / 100;
let totalConPropina = costoComida + valorPropina;

console.log("Costo de la comida: $" + costoComida);
console.log("Propina (10%): $" + valorPropina);
console.log("Total a pagar: $" + totalConPropina);
```

---

## 3. Tipos de Datos Primitivos

| Tipo             | Descripción                           | Ejemplo               |
|------------------|---------------------------------------|-----------------------|
| <b>Number</b>    | Valores numéricos enteros y decimales | 42 , 3.14             |
| <b>BigInt</b>    | Enteros de longitud arbitraria        | 12345678901234567890n |
| <b>Boolean</b>   | Estados lógicos: true o false         | true , false          |
| <b>Undefined</b> | Variable declarada sin valor asignado | let x;                |

| Tipo   | Descripción                     | Ejemplo                                                         |
|--------|---------------------------------|-----------------------------------------------------------------|
| Null   | Ausencia intencional de valor   | <code>let x = null;</code>                                      |
| Symbol | Valor único e inmutable         | <code>Symbol('id')</code>                                       |
| String | Cadenas de texto entre comillas | <code>'Hola'</code> , <code>"Hola"</code> , <code>`Hola`</code> |

## Strings: Comillas Simples, Dobles y Backticks

```
let nombreEstudiante = "Juanito";
let apellidoEstudiante = "Bueno";
let ciudad = "Medellín";

// Interpolación con backticks
let mensajeBackTicks = `Bienvenido ${nombreEstudiante}
${apellidoEstudiante} desde ${ciudad} usando interpolación con
backticks`;
console.log(mensajeBackTicks);

// Concatenación con +
let mensajeConcatenado = 'Bienvenido ' + nombreEstudiante + ' ' +
apellidoEstudiante + ' desde ' + ciudad + ' usando concatenación';
console.log(mensajeConcatenado);
```

| Sintaxis            | Nombre           | Permite variables dentro          |
|---------------------|------------------|-----------------------------------|
| <code>'Hola'</code> | Comillas simples | No                                |
| <code>"Hola"</code> | Comillas dobles  | No                                |
| <code>`Hola`</code> | Backticks        | Sí, con <code>\${variable}</code> |

## Ejemplo: Generador de Correo Electrónico

```
let primerNombre = "Ana";
let primerApellido = "Pérez";
let dominio = "empresa.com";

let primeraLetra = primerNombre[0]; // Tomo el primer carácter
(índice 0)
```

```
let correo = primeraLetra + primerApellido + "@" + dominio;
let correoMinuscula = correo.toLowerCase();

console.log("Nombre:", primerNombre, primerApellido);
console.log("Correo generado:", correoMinuscula);
```

## Métodos Útiles para Strings

```
let texto = "  Hola Mundo  ";

console.log("Original: '" + texto + "'");
console.log("Sin espacios: '" + texto.trim() + "'");
console.log("Mayúsculas: " + texto.toUpperCase());
console.log("Minúsculas: " + texto.toLowerCase());
console.log("¿Contiene 'Mundo'? " + texto.includes("Mundo"));
console.log("Longitud: " + texto.length);
```

| Método         | Descripción                        |
|----------------|------------------------------------|
| .trim()        | Elimina espacios al inicio y final |
| .toUpperCase() | Convierte a MAYÚSCULAS             |
| .toLowerCase() | Convierte a minúsculas             |
| .includes()    | Busca texto dentro de la cadena    |
| .length        | Cuenta los caracteres              |

---

## 4. Boolean y Operadores de Comparación

```
let edad = 15;
let puedeEntrar = edad >= 18;

console.log("Edad:", edad);
console.log("¿Puede entrar?", puedeEntrar);
```

### Operadores de Comparación

| Operador | Significado                      |
|----------|----------------------------------|
| >        | Mayor que                        |
| <        | Menor que                        |
| <=       | Menor o igual que                |
| >=       | Mayor o igual que                |
| ===      | Igualdad estricta (valor y tipo) |
| !==      | Diferente                        |

```

console.log("5 > 3:", 5 > 3);           // true
console.log("5 < 3:", 5 < 3);           // false
console.log("5 >= 5:", 5 >= 5);         // true
console.log("5 === 5:", 5 === 5);       // true
console.log("5 === '5':", 5 === '5');   // false (número vs texto)

```

## Operadores Lógicos

| Operador | Significado              | Ejemplo               |
|----------|--------------------------|-----------------------|
| &&       | Y (ambos deben ser true) | true && false → false |
|          | O (al menos uno true)    | true    false → true  |
| !        | NO (invierte)            | !true → false         |

```

console.log("true && true =", true && true); // true
console.log("true && false =", true && false); // false
console.log("true || false =", true || false); // true
console.log("false || false =", false || false); // false
console.log("!true =", !true); // false
console.log("!false =", !false); // true

```

## Ejemplo: Sistema de Descuento por Membresía

```

let esMiembroPremium = true;
let totalCompra = 150000;

```

```
let descuentoPremium = esMiembroPremium;
let descuentoPorMonto = totalCompra > 100000;

console.log("¿Es miembro premium?", esMiembroPremium);
console.log("¿Compra mayor a 100mil?", descuentoPorMonto);

// OR (||) - Basta con que una condición sea true
let aplicaAlgunDescuento = descuentoPremium || descuentoPorMonto;
console.log("¿Aplica algún descuento?", aplicaAlgunDescuento);

// Descuento especial del 30%: necesita ambas condiciones
let descuentoEspecial = esMiembroPremium && (totalCompra > 100000);
console.log("¿Aplica descuento especial?", descuentoEspecial);

// Usando NOT (!) para preguntar lo contrario
let noAplicaDescuento = !descuentoEspecial;
console.log("¿No aplica el descuento especial?", noAplicaDescuento);
```

---

## 5. Estructuras de Datos: Arreglos (Arrays)

Un **arreglo** es un tipo especial de objeto utilizado para almacenar listas ordenadas de elementos bajo una sola variable.

```
let listaDeCompras = ["pan", "leche", "chocolate", "huevos", "queso"];

console.log("Lista de compras", listaDeCompras);
console.log("Primer elemento (índice 0): ", listaDeCompras[0]);
console.log("Segundo elemento (índice 1): ", listaDeCompras[1]);
console.log(`Tercer elemento: ${listaDeCompras[2]}`);

console.log("Cantidad de items:", listaDeCompras.length);

let ultimoIndice = listaDeCompras.length - 1;
console.log("Último elemento:", listaDeCompras[ultimoIndice]);
```

### Métodos para Modificar Arreglos

```
// push() - Agrega un elemento al FINAL
listaDeCompras.push("café");
console.log("Después de push('café'):", listaDeCompras);
console.log("Nueva longitud:", listaDeCompras.length);

// pop() - Elimina el ÚLTIMO elemento y lo devuelve
let itemEliminado = listaDeCompras.pop();
console.log("Eliminé:", itemEliminado);
console.log("Después de pop():", listaDeCompras);

// unshift() - Agrega un elemento al PRINCIPIO
listaDeCompras.unshift("jugo");
console.log("Después de unshift('jugo'):", listaDeCompras);

// shift() - Elimina el PRIMER elemento
let primerItem = listaDeCompras.shift();
console.log("Eliminé el primero:", primerItem);
console.log("Después de shift():", listaDeCompras);
```

| Método     | Acción                            |
|------------|-----------------------------------|
| .push()    | Agrega al final                   |
| .pop()     | Elimina del final                 |
| .unshift() | Agrega al principio               |
| .shift()   | Elimina del principio             |
| .length    | Devuelve la cantidad de elementos |

## Ejemplo: Lista de Tareas con Estado

```
let tareas = [
  { nombre: "Estudiar JavaScript", completada: false },
  { nombre: "Hacer ejercicio", completada: true },
  { nombre: "Comprar comida", completada: false },
  { nombre: "Llamar a mamá", completada: false }
];

console.log("=== MIS TAREAS ===");
console.log("Tarea 1:", tareas[0].nombre, "- Completada:",
```

```
tareas[0].completada);
console.log("Tarea 2:", tareas[1].nombre, "- Completada:",
tareas[1].completada);
console.log("Tarea 3:", tareas[2].nombre, "- Completada:",
tareas[2].completada);
console.log("Tarea 4:", tareas[3].nombre, "- Completada:",
tareas[3].completada);

// Agregar una tarea nueva al arreglo
let nuevaTarea = { nombre: "Leer un libro", completada: false };
tareas.push(nuevaTarea);
console.log("Nueva tarea agregada:", tareas[4].nombre);

// Marcar la primera tarea como completada
tareas[0].completada = true;
console.log("Actualicé tarea 1:", tareas[0]);

console.log("Total de tareas:", tareas.length);
```

---

## 6. Estructuras de Datos: Objetos (Objects)

Un **objeto** es una estructura de datos que guarda información en pares **clave: valor** (key: value).

A diferencia de los arreglos que usan índices numéricos, los objetos usan nombres (claves) para acceder a los valores.

- Las claves son strings.
- Los valores pueden ser cualquier tipo de dato: number, string, boolean, array, otro objeto.
- Los objetos se definen con llaves `{ }`.
- Para acceder a un valor: `objeto.clave` (notación de punto) o `objeto["clave"]` (notación de corchetes).

### Ejemplo: Perfil de Usuario

```
let usuario = {
  nombre: "Ana",
```

```
    edad: 25,
    ciudad: "Bogotá"
};

console.log("Objeto completo:", usuario);

// Notación de punto
console.log("Nombre (notación punto):", usuario.nombre);
console.log("Edad (notación punto):", usuario.edad);

// Notación de corchetes
console.log("Ciudad (notación corchetes):", usuario["ciudad"]);

// Modificar una propiedad existente
usuario.edad = 26;
console.log("Edad actualizada:", usuario.edad);

// Agregar una nueva propiedad
usuario.profesion = "Programadora";
console.log("Después de agregar profesión:", usuario);

// Eliminar una propiedad
delete usuario.ciudad;
console.log("Después de eliminar ciudad:", usuario);
```

## Ejemplo: Objeto con Diferentes Tipos de Datos

```
let producto = {
    nombre: "Laptop Gamer",
    precio: 2500000,
    disponible: true,
    etiquetas: ["electrónica", "computadores", "oferta"],
    especificaciones: {
        marca: "Lenovo",
        ram: "16GB",
        almacenamiento: "512GB SSD"
    }
};

console.log("=== PRODUCTO ===");
```

```
console.log("Nombre:", producto.nombre);
console.log("Precio: $" + producto.precio);
console.log("Disponible:", producto.disponible);
console.log("Primera etiqueta:", producto.etiquetas[0]);
console.log("Marca:", producto.especificaciones.marca);
console.log("RAM:", producto.especificaciones.ram);
```

---

## 7. Resumen de Tipos de Datos

| Tipo    | Declaración                              | Acceso         | Métodos/Operadores                               |
|---------|------------------------------------------|----------------|--------------------------------------------------|
| Number  | let precio = 1500;                       | —              | + - * / % **                                     |
| String  | let nombre = "Ana";                      | nombre[0]      | .toLowerCase(), .toUpperCase(), .trim(), .length |
| Boolean | let puedeEntrar = edad >= 18;            | —              | &&, \ \ , !                                      |
| Array   | let lista = ["pan", "leche"];            | lista[0]       | .push(), .pop(), .length                         |
| Object  | let usuario = {nombre: "Ana", edad: 25}; | usuario.nombre | delete, asignación directa                       |